

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 04: **Software Engineering:** ***Perforce Configuration*** ***for the CW***

Tutorial Overview

In today's tutorial we will learn to use the basic features of Perforce source control management system.

NOTE: This tutorial can only be completed once you have access to the Perforce Server and have been assigned to your group depot – this will happen at the end of Week 1.

- The **First Task** is to download and install Perforce Visual Client (P4V). Can skip this task in the lab.
- The **Second Task** is to run P4V and connect to the perforce server.
- The **Third Task** is to create a Workspace.
- The **Fourth Task** is to get latest to keep the workspace up to date.
- The **Fifth task** is adding new files to the depot.
- The **Sixth task** is to submit the changelist.
- The **Seventh task** is to diff the changelist file and repository file.
- The **Eighth task** is to add your assignment project to source control.
- The **Ninth task** is to set up your project for Visual Studio sharing and making sure everyone has an up-to-date synchronized copy.
- The **Tenth task** is to make sure you are comfortable with modifying code within Visual Studio.
- The **Additional task** is Handling conflicts.

Task 1: Download and install Perforce Visual Client (P4V)

This is a task to be completed only if you have never installed Perforce on your personal development machine off campus. All School laboratory machines have P4V installed – it is recommended that you complete this tutorial both on campus and off campus so you can work on the coursework in your independent study time.

You will need to download and install Perforce Visual Client (P4V) at home.

If you are using the VPN to connect to the Lab machines to work from home; P4V is already installed on the lab image so you can skip to Task 2.

Step 1: Download P4V

Download P4V from the following website. Select a version appropriate for your home operating system.

<https://www.perforce.com/downloads/helix-visual-client-p4v>

Step 2: Install P4V

Run the executable and follow the instructions to install P4V on your home computer.

Task 2: Run P4V and connect to the perforce server

Step 1: Run P4V

Search for P4V on the start menu and run the application (Press the Windows Key and type P4V is the easiest way).

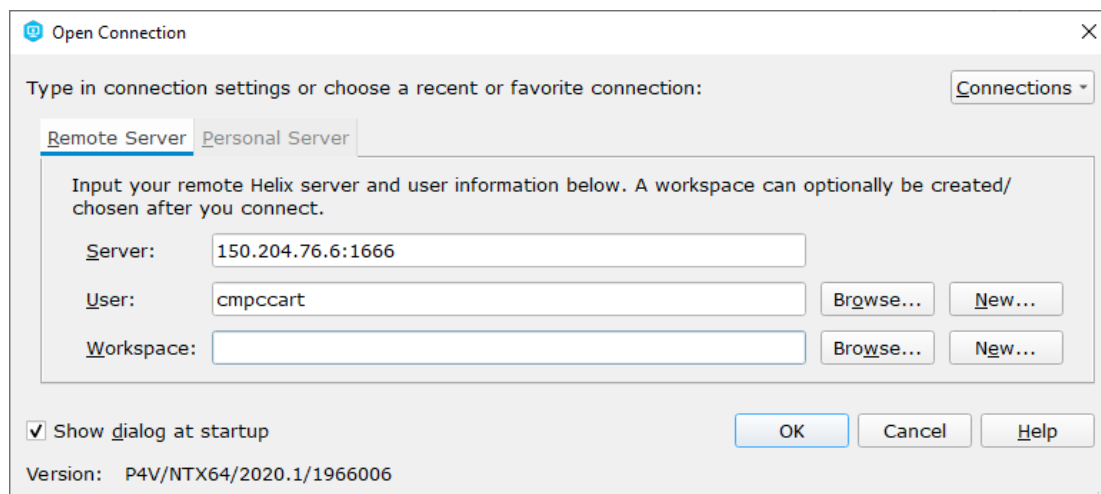
Step 2: Connect to the server

On the open connection window, select the Remote Server tab and provide the following information:

Server: 150.204.76.6:1666 (NOTE: from within the labs, the default of perforce:1666 will work, but this will not from home).

User: <YOUR LJMU USERNAME> - this will be in lowercase for most users.

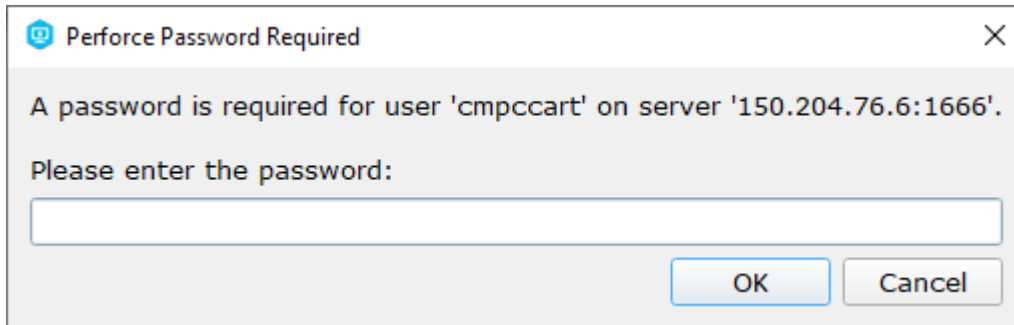
As in the example below:



This information should be stored on the computer, so you should only have to type out the IP address once, on your home machine.

Click on OK.

Step 3: Enter your password



You will be prompted to enter your password. This is the same as the password as you use to log onto university computers as we authenticate using the same servers.

You will be prompted for your password every time you connect to the Perforce server.

Click on OK.

Task 3: Creating a Workspace for your Group

NOTE: If you have yet to confirm your group with the module leader, you won't be able to do this yet.

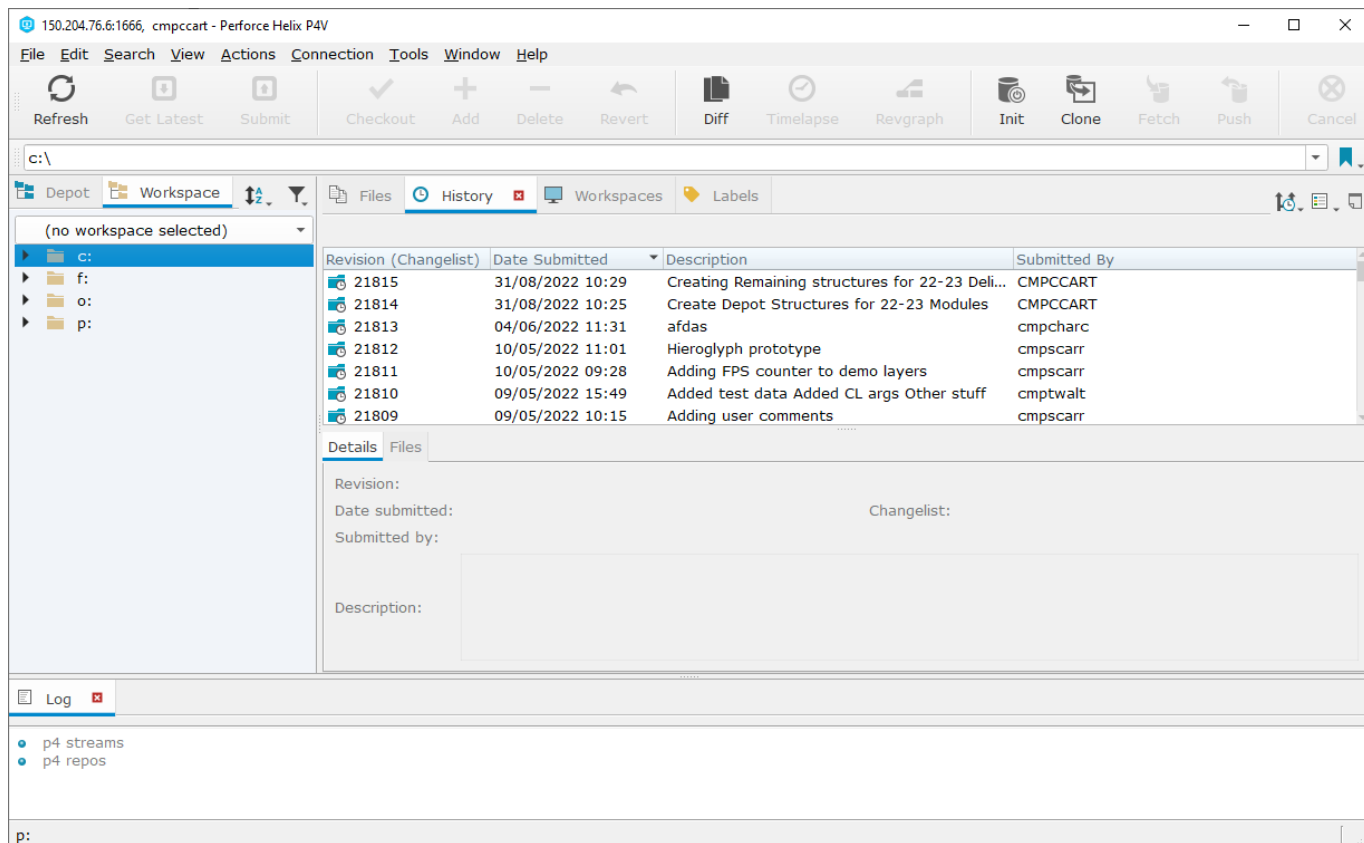
This should be your task for today – try and form a group ASAP and let the module leader know – he can then set your Perforce Depot up for you.

It is best practice to have a single Workspace for a single Depot Location, if possible – that is what we will achieve today. You will need to set up a workspace in order to store a local version of the repository on your computer.

You will need a new workspace for each computer that you use to work on this project.

Step 1: P4V Interface

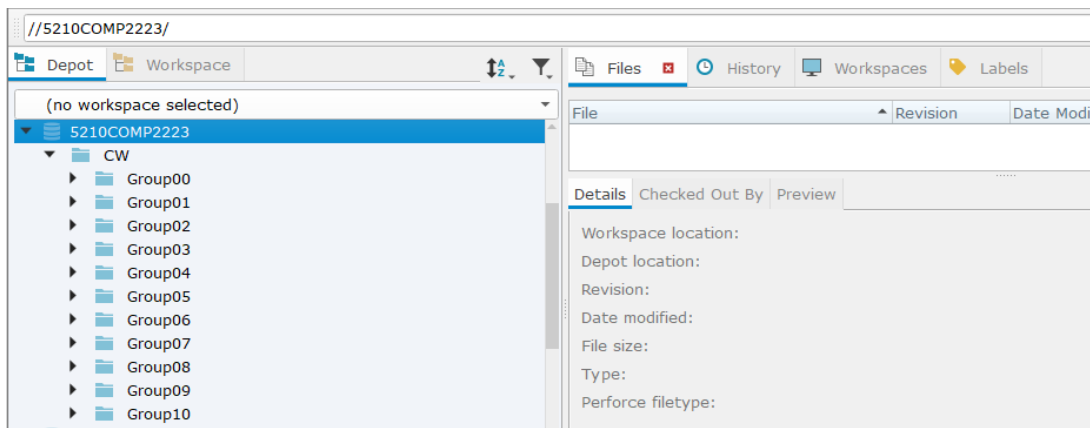
Once logged in you will be presented with the P4V interface:



Step 2: Verify what Depots you have access to.

The depot view on the left-hand side will show you the Depots on our Server that you have permission to access – You should have access to **5210COMP2324**.

Depots are projects that are stored on the server – we need to choose which depots should be synchronized locally – this is what gives us a workspace.

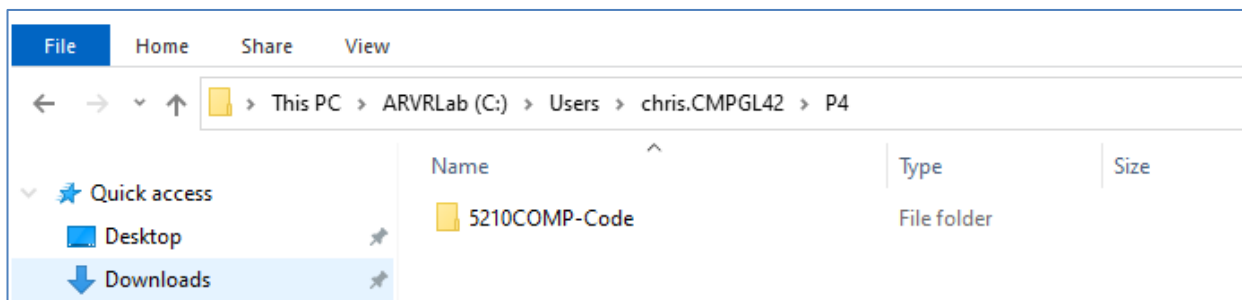


Step 3: Create a new workspace

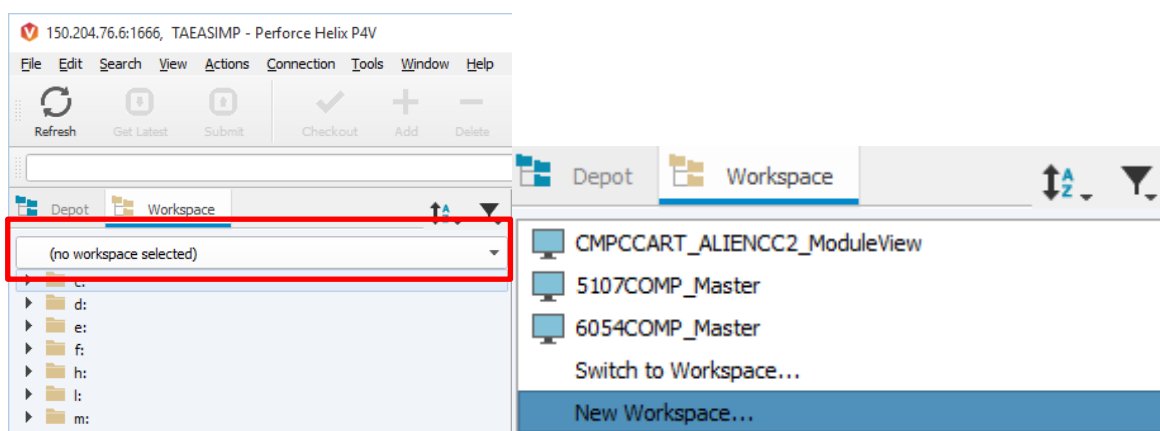
It is suggested that you store your Perforce files somewhere in your personal storage area under Windows.

Open your C:\users\<username> folder (e.g., C:\users\cmpjblog if your username is **cmpjblog**) in Windows Explorer and create a new folder called **P4**.

Inside that, create a new folder called *5210COMP-Code*.



Back in P4V, On the workspace tab, select the “(no workspace selected)” dropdown menu and select “New Workspace...”



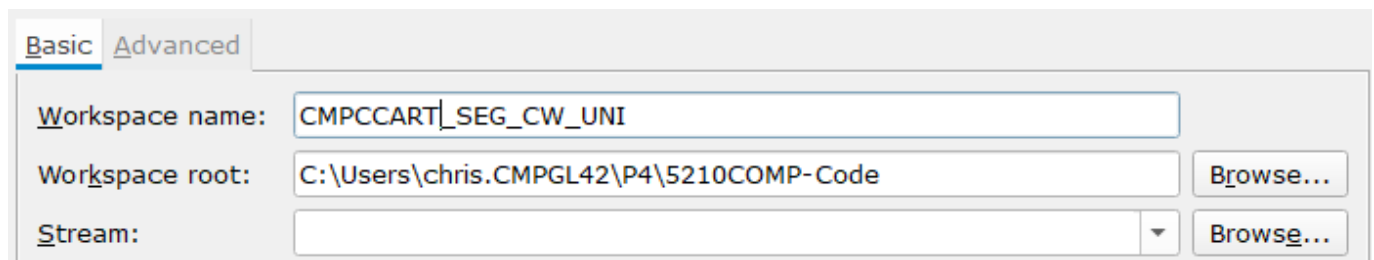
You will be presented with a New Workspace dialogue.

Fill in the following information (where USERNAME is your username, e.g. cmpccart or CC).

Workspace name: USERNAME_SEG_CW_UNI

The reason for this is because **each workspace must be uniquely named across all devices** – we are naming it by **user-module-type-location**. Therefore, at home, you will change the location from UNI to HOME, for example.

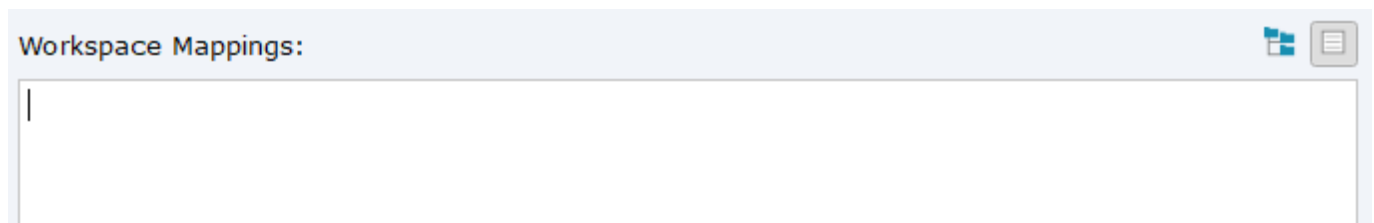
Next, point your Workspace Root to the folder we created (5210COMP-Code), using the *browse* option.



Locate the workspace mapping and switch to Text view by clicking the far-right icon.



Next, **Delete Everything** that is already inside the window:



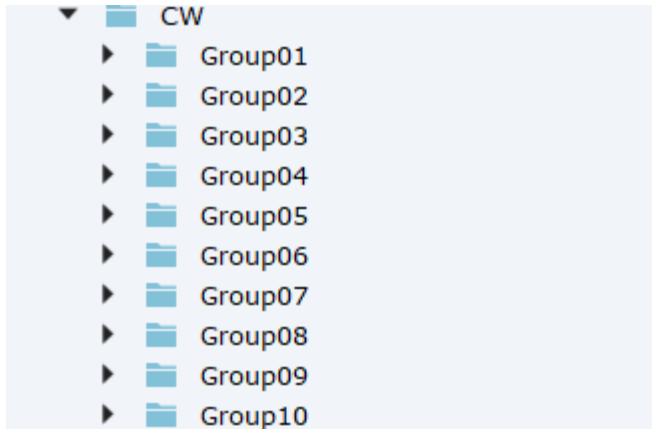
Crucially, this time, we are not going to map anything here, Just Click 'OK'.

In the main window of P4V, Switch Tab to the **Depot Tab** and Find the **5210COMP2324** Depot.

If you have set up your group, you will be mapped to a group folder, with the same number as your group on Canvas (see the Canvas Groups under People to see your Group Number).

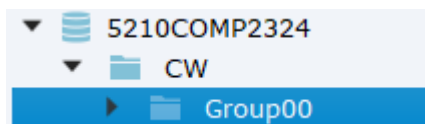
I have created a page which Maps this for you as well, which you can see under the Perforce Section under the Working from Home area.

You should have something like the following:

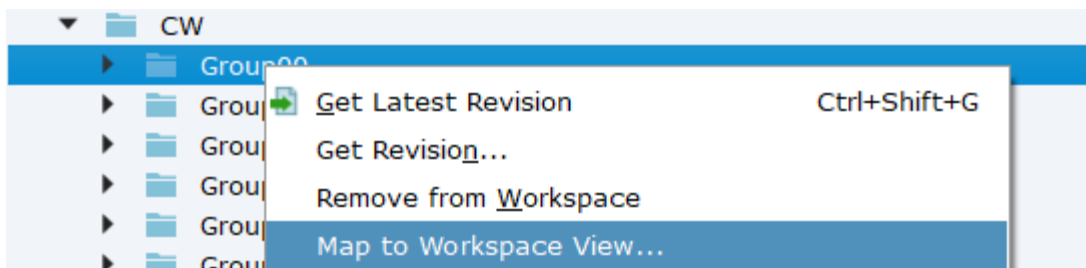


You should only be able to see one group – the one you are a confirmed member of.

In this demo, I am a member of group **Group00**



Right Click on your Group Folder (e.g., Group00) and select '**Map to Workspace View**':



You can see that it generates the Mapping for us.



However, there is a couple of little problems:

- Firstly, make sure that it is also on the first line of this window (i.e., there are no blank lines above it) – you can see it has left two blank lines in the sample above.
- By default, it puts the sub folders into the Workspace as well

Workspace Mappings:

```
//5210COMP2324/CW/Group00/... //CMPCCART_SEG_CW_UNI/5210COMP2324/CW/Group00/...
```

The folder /5210COMP2324/CW is unnecessary, as is its sub folder Group00 – we want our Code folder to contain our **Depot content**, not more sub-folders.

The reason for this is to cut down on potential issues - if we had these sub folders and then put files in CW, or files in the workspace itself, they are outside the view of our project:

- The folder CW would map to **//5210COMP2324/CW**, which you don't have access to, since this would mean you can access all groups.
- The workspace folder would map to **//5210COMP2324/** you also don't have access to since this would mean access to all files in this Depot.

Therefore, we don't want this – By making the Root of the Workspace map to CW/Group00, there is no folder I can put files in which is outside the server's view – since the folder above the workspace is not known by Perforce at all.

So, we need to remove everything after the **Workspace Name** and the Triple Dots (...) (e.g., I need to remove **5210COMP2324/CW/Group00/**) and you should end up with something like:

Workspace Mappings:

```
//5210COMP2324/CW/Group00/... //CMPCCART_SEG_CW_UNI/...
```

I CANNOT STRESS HOW IMPORTANT IT IS TO GET THIS MAPPING CORRECT – WITHOUT IT YOUR CODE MANAGEMENT SIMPLY WILL NOT FUNCTION.

Click 'OK' to Create the Workspace.

SO, WHY HAVE WE USED THIS METHOD?

MAPPING TO THE WORKSPACE VIEW GUARANTEES THAT THE DEPOT LOCATION IS SPECIFIED CORRECTLY – THIS CAN BE DIFFICULT IF IT IS A COMPLEX PATH.

In the next task, we will synchronize this workspace.

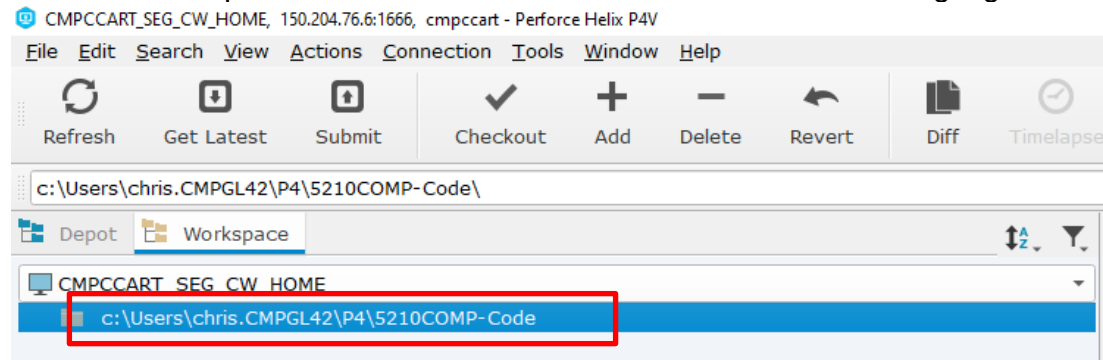
Task 4: Getting Latest Version.

To download the latest versions of files stored in the repository we must “Get Latest” on our workspace. There are already files in each group’s directory, so we will now synchronise the depot with our workspace we created in Task 3.

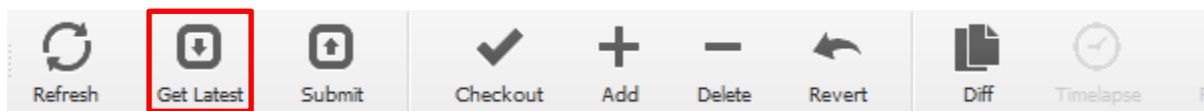
Step 1: Get Latest on your workspace

On the workspace tab, you should now be able to see your local version of the repository. You can tell which Workspace you are working in by looking at the application bar – it will display your workspace name. Currently your folder will be empty, this is because you need to get the latest files from the repository.

In the Workspace tab, select the folder so that it is highlighted :

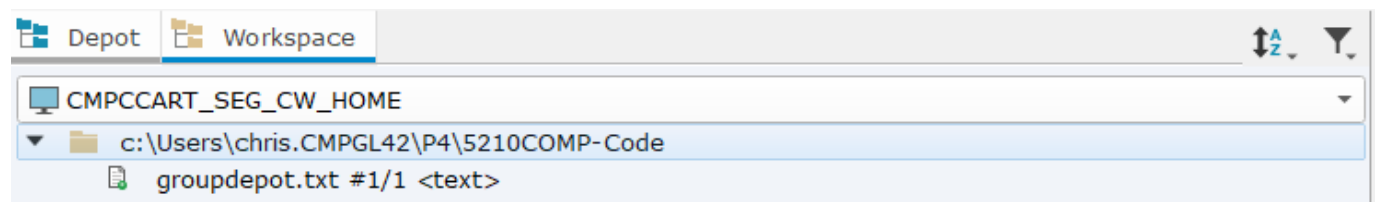


We could do it the same as before using the right click, but we can also do this from the menu bar, by selecting the ‘Get Latest’ icon:



You will need to “Get Latest” every time you want to get the latest versions of files from the repository.

If you have done this successfully, you should see a new text file in your folder. Otherwise, check your settings again.



I have added a text file to each Group Depot, which should now be in your workspace.

If you can't see any team folder AND YOU HAVE CONFIRMED YOUR GROUP MEMBERSHIP or, you can see the wrong team folder then please let me know asap!

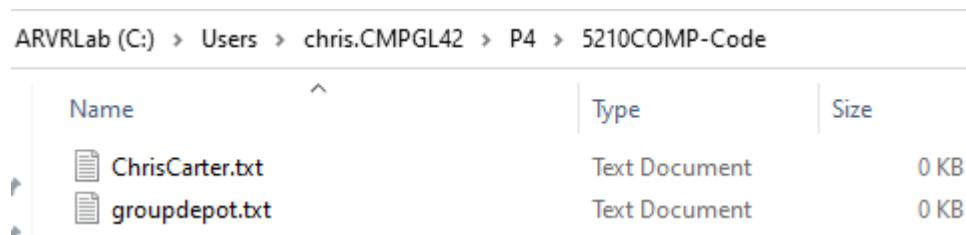
Task 5: Adding new files to the depot – each group member should attempt this!

During your assignment you will need to add new files to the depot (e.g. your new code!)

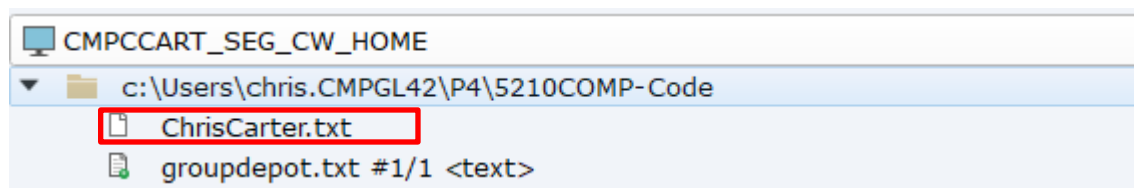
Step 1: Create a new text file

Start by right clicking on your Workspace folder and selecting 'Show In -> Show In Explorer' which is near the bottom of the menu. This is a really easy way of going to the folder in Windows.

In your workspace directory, use the window's right click menu to create a new text file with <YourName>.txt as the filename inside this folder:



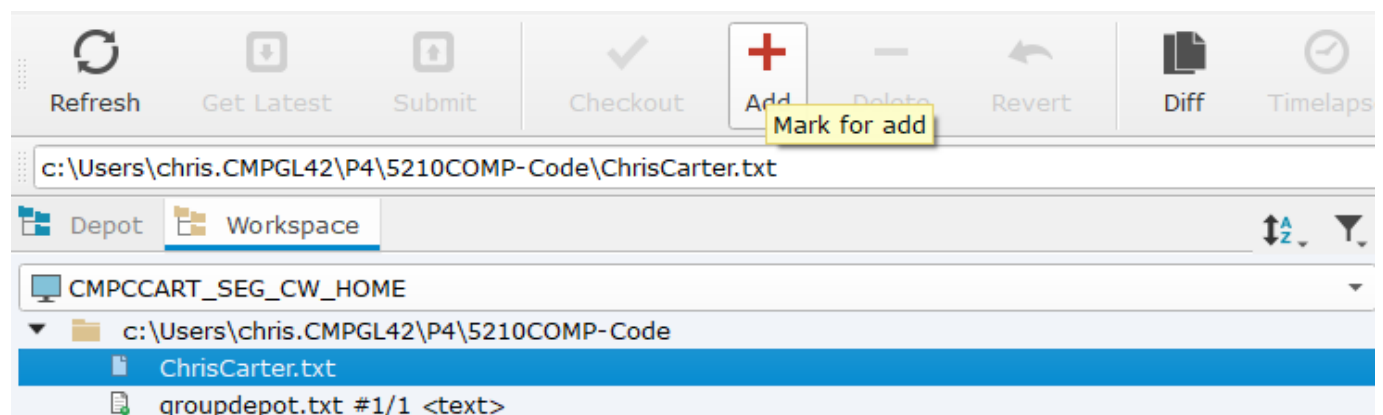
Go back to P4V; In the workspace tab, press F5 or the refresh button, you should now see your new files on the client workspace.



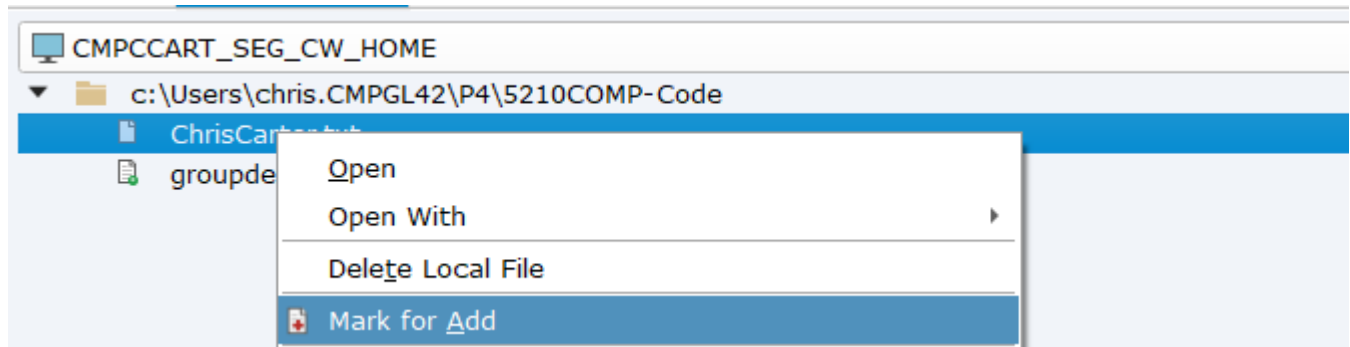
Notice that the icon is different to the other file and there is no information after the file name – this is because whilst the file is in the directory, it is not synched with Depot server.

Step 2: Add the file to a changelist

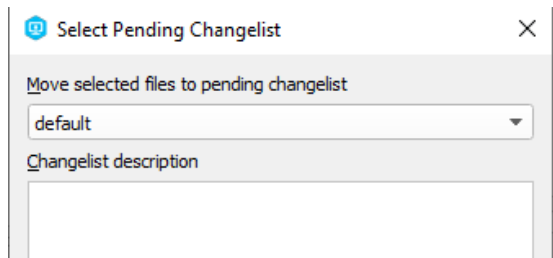
In order to add the file into the repository, right click on the file(s) (or folder(s)) and select "Mark for Add" or click the **Add** Button – this will be greyed out if the file is already on the server:



OR:

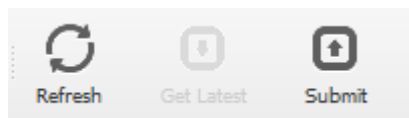


This will add the file(s) to the **default pending** changelist after you click OK on the Dialogue (sometimes it will do this automatically):

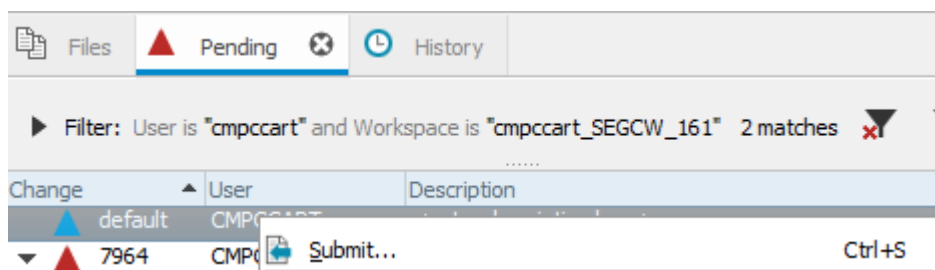


Step 3: Submit the changelist

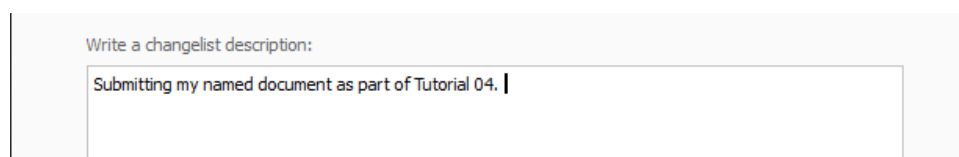
To submit the changelist, either left click on the default change list (under the pending tab) and select 'Submit' from the Menu Bar:



OR right-click on the changelist under the **Pending** view, and click submit:



You will be presented with a "Submit Changelist" dialogue where you are prompted to write a changelist description:



Whenever you submit a changelist, you should always provide a good changelist description that explains the changes that you have made so that other team members can understand what the submission is supposed to do.

I will be assessing you on this when I mark the coursework.

TASK:

- Write a Comment which states you are submitting your named document as part of completing Tutorial 04.

You can review the files that are part of the changelist and use the top left icon to select / deselect all files if there are lots of files in the submission.

TASK:

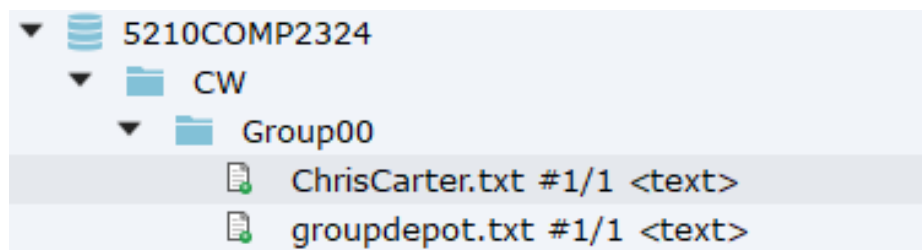
- Click the Submit button to submit the file to the Depot. Depending on how big the files are, you may get a progress bar – in this case you shouldn't.

After completing your submission, your file will now be under source control.

Every version of the file that you submit will be stored, changes can be tracked and reverted to an older version if necessary.

Make sure that everyone can see your new file - All your team members can now perform a "Get Latest" and obtain your changes from the Depot – this is how we synchronize as a team in Perforce.

You can also inspect the Depot View and see what files are present to make sure it has been correctly uploaded.

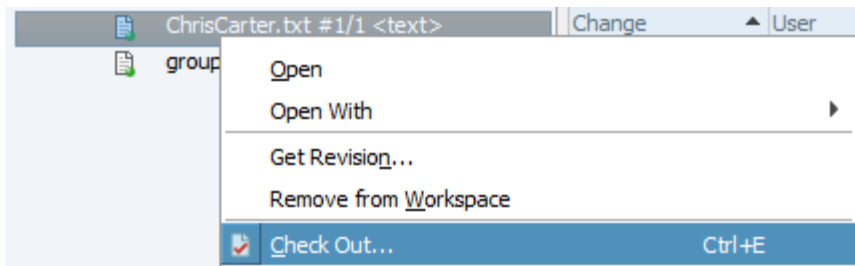


Task 6: Modifying files under source control

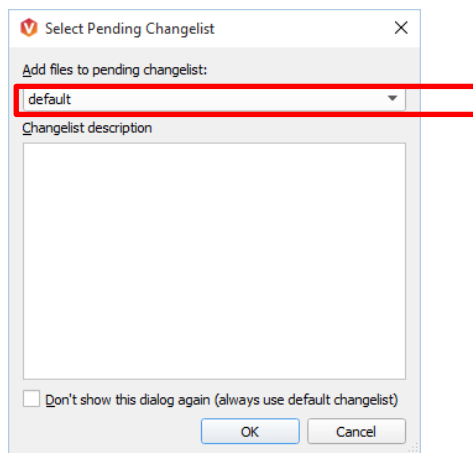
Files under source control will become **read only**, so they cannot be modified by accident. To edit the files again we must “**Check out**” the file from Perforce (think of it like having a bookshelf – we must check the file (book) out from the library before we can work with it).

Step 1: Check out <your name.txt>

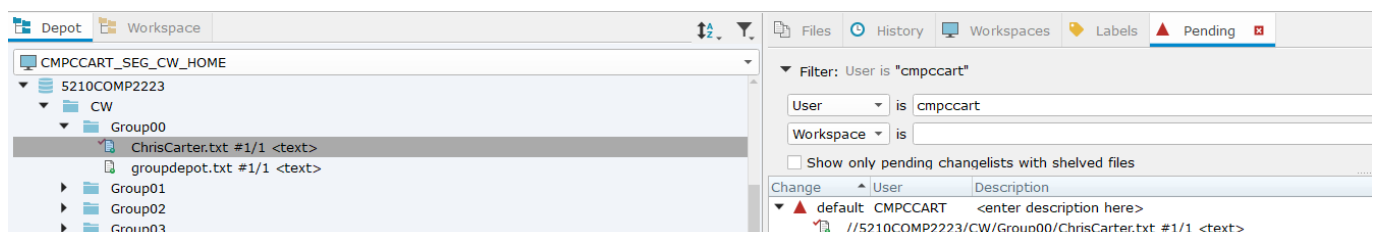
In order to check out a file, locate it in the workspace tab, right click on the file(s) and select “Check Out...”.



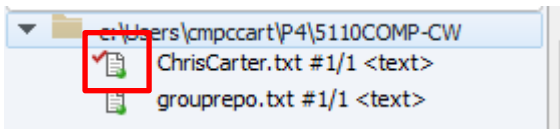
You will be prompted to select which changelist to add the checked-out file to. Leave this on default and select OK:



The file will now appear on the Pending Changelists tab under the default changelist:

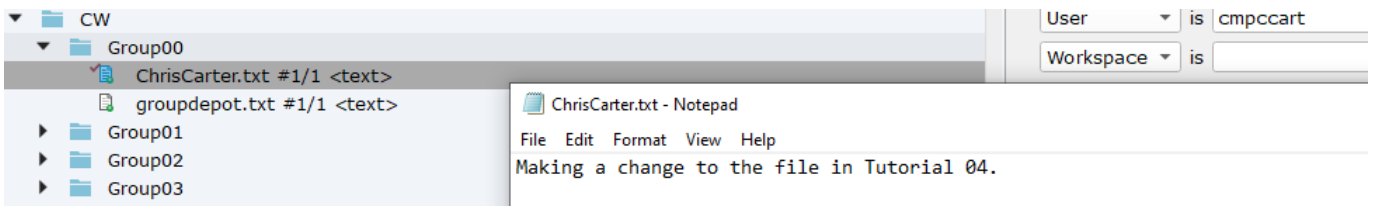


A red tick on the file in the Depot/Workspace tab as well as in the pending changelist will signify that the file is being checked out from source control:



Notice that the other file hasn't. You are now free to edit the file that you have checked out. You can do this directly in Windows, or by right clicking on the file and selecting Open – this will use the default editor for that file type (e.g., a text file is typically notepad).

- Modify the file and add some text:



Save the file in your text editor – this means the workspace has a new version of the file, but it is not yet on the server.

Task 7: Diff the changelist file and repository file

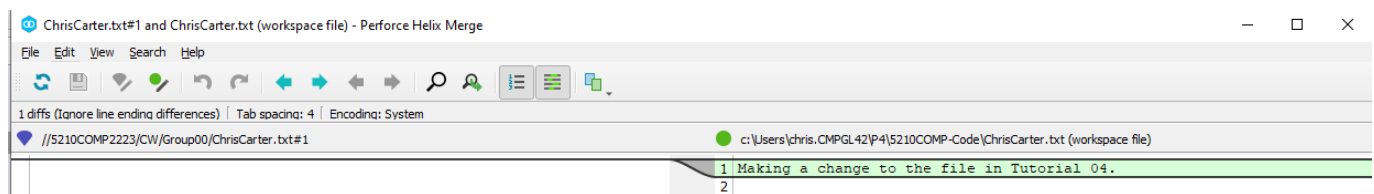
Before submitting a file back into Perforce, you should always diff (*check the difference between the modified file and repository file*) the file to check that you are not submitting some temporary code by accident.

This should be completed with another programming team member as a sanity check. We call this a [Code Review](#)

Step 1: Launch the diff tool

In order to diff the file, select the file in the pending changelist tab and press Ctrl + d or right click and select “Diff Against Have Revision”:

This will launch the P4 merge tool:

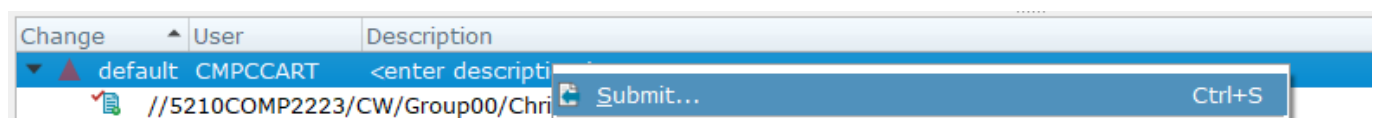


The Depot’s version of the file will be displayed on the left and the local file will be displayed on the right.

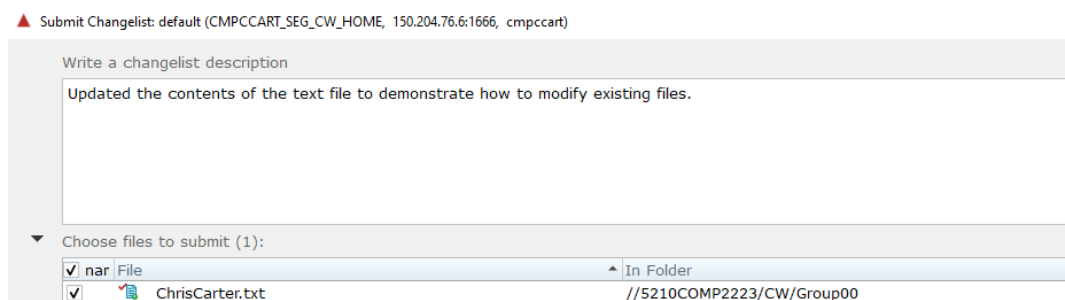
Any changes in the file will be highlighted in green. You can see in the example above; I have added a line to the file with some new text.

If you are happy with the changes and what the diff tool shows you, you can close the merge tool and submit the file, otherwise, have a look at the other options in the Merge tool first.

Easiest way to submit the correct files is to use the changelist view itself – simply right click on the default change list and then choose to submit.



Again, complete a comment about what you have changed in the file so that we have an accurate log of the work we have done.



Task 8: Adding your current project to source control (Group Task).

As a team you should add your project to source control.

We want to add any files that are required to build the project, but none of the auto-generated files.

Pick the Group member who has made the most progress so far to commit the initial version (ideally the completed version of Tutorial 03).

You should start by copying your code for this inside your **5210COMP-Code** folder.

NOTE: The Outer Folder of the engine project (SEG_RelicApp_Template when you download it) can be named whatever you want it to be – *it is only important that the folders inside it have the same structure.*

In this case, I have renamed it to reflect my group name and module code – Software Engineering for Games = SEG so SEG_Group00:

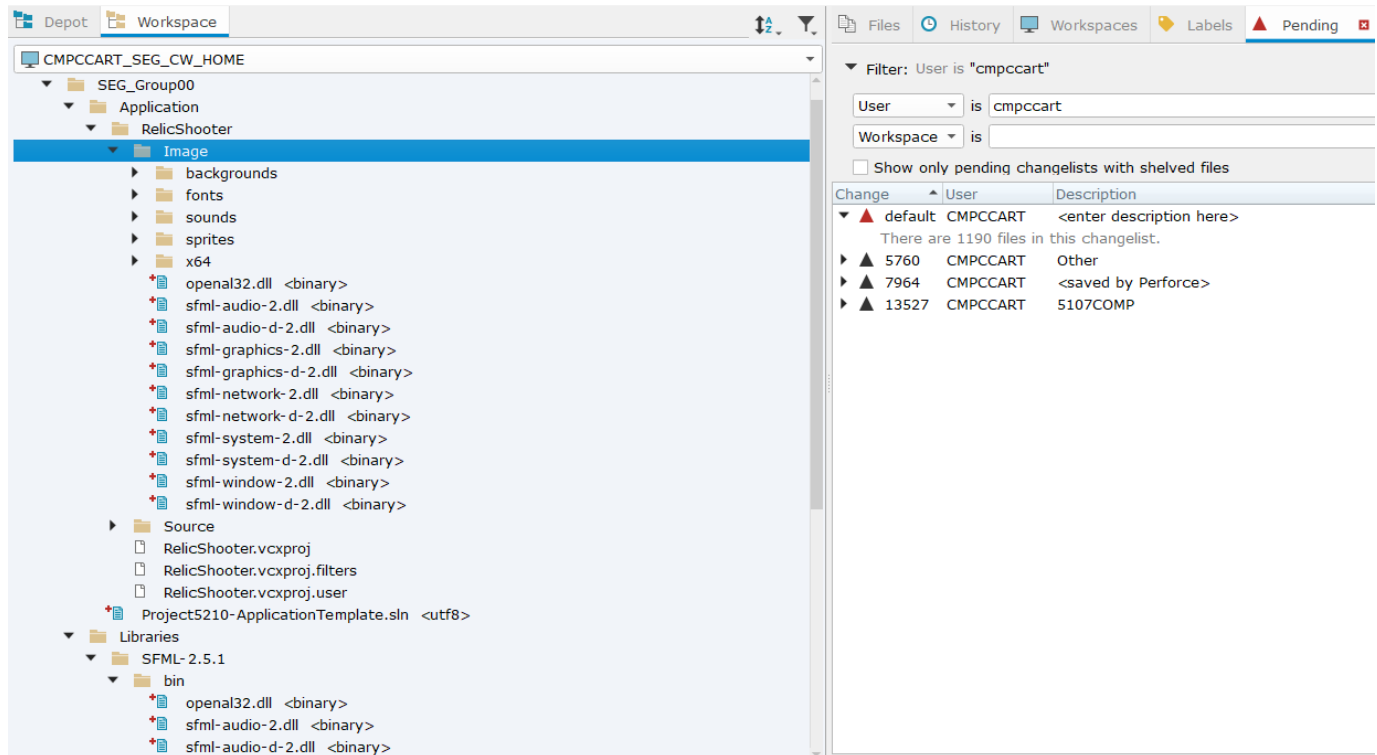
Mark the following files for add in Perforce:

- The **Libraries** folder (all files in this folder are fine).
- Project5210-ApplicationTemplate.sln (Inside the Application/RelicShooter folder) – **you can rename this .sln file if you like – it may have a different name to this depending on which tutorial you are using.**
- The **Image** Folder (Inside the Application/**RelicShooter** folder)
- The **Source** Folder (Inside the Application/**RelicShooter** folder)

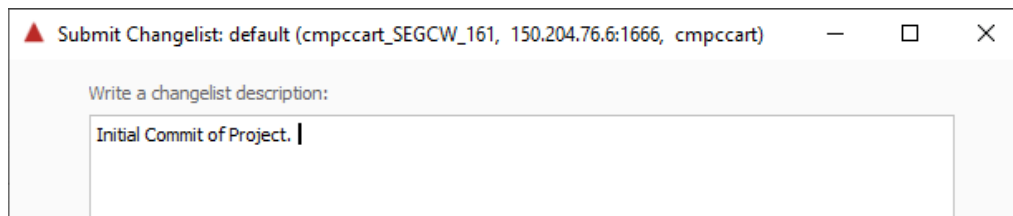
DO NOT add the following (yet):

- Any Debug folder
- Any Release folder
- Assets folder
- The Visual Studio files (.vcxproj etc).

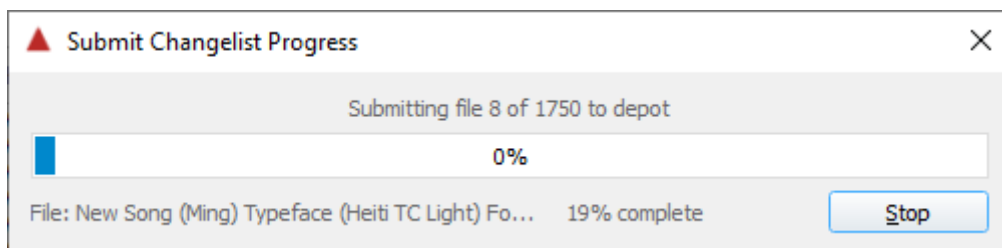
Before you submit, you should have something like the following:



Submit your files using the Default Changelist and add an appropriate comment (e.g. initial commit of application code).

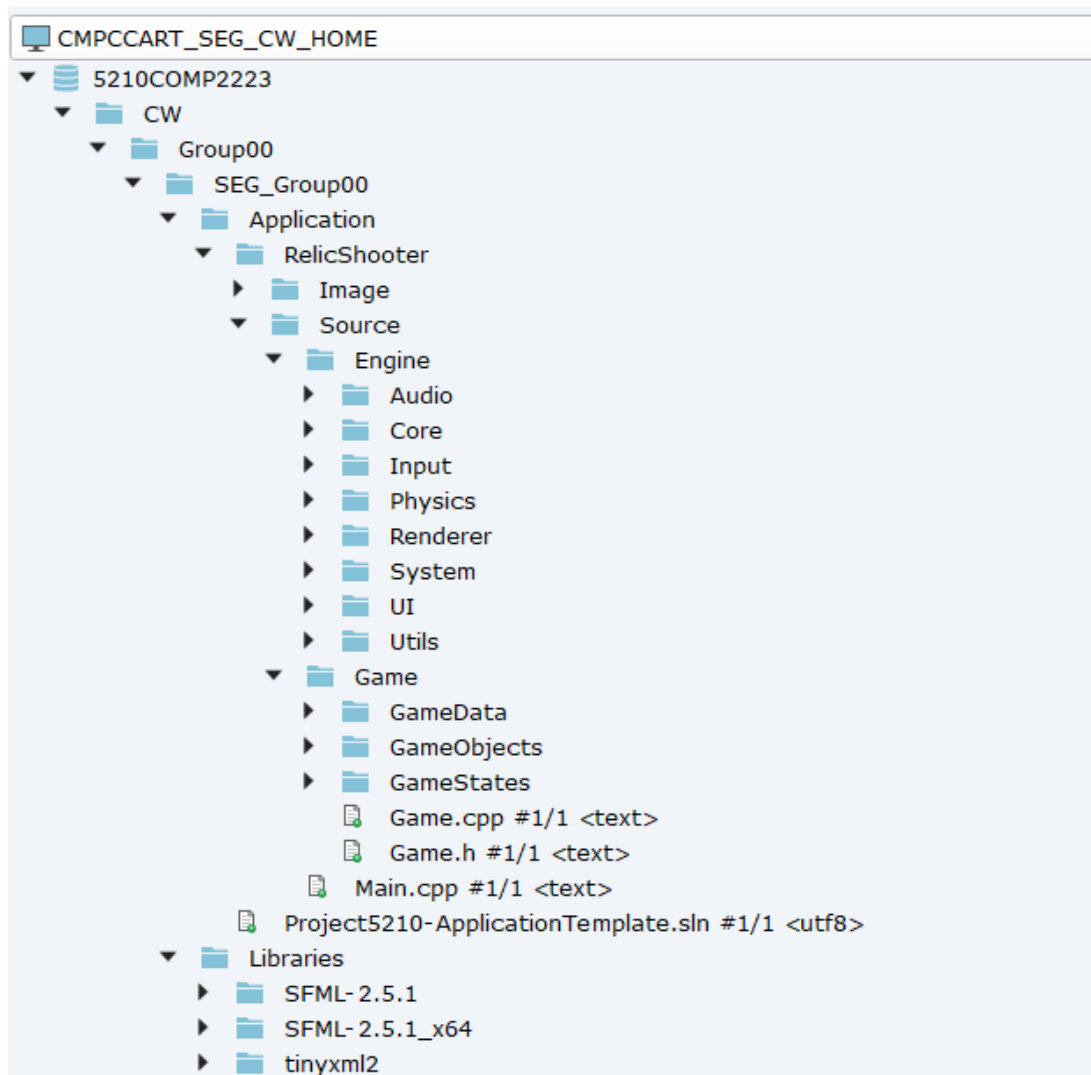


This step may take a while, but it means that any subsequent commit should be quick, as you have the libraries and most the assets up on the Depot.



Notice that it is only files that get the Icons, not the folders – if a folder has no files committed (e.g. Assets), then it will not appear in the Depot view.

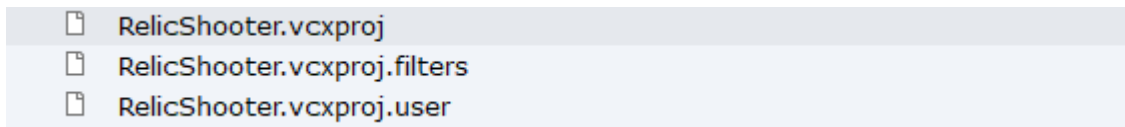
Once committed, your file structure should resemble something like the following (*don't worry if there are slight differences with the file names*):



We have submitted the files, to allow us to begin working as a team! We also need to work with Visual Studio as well.

Task 9: Working with Visual Studio and Making sure your group can all work with the code.

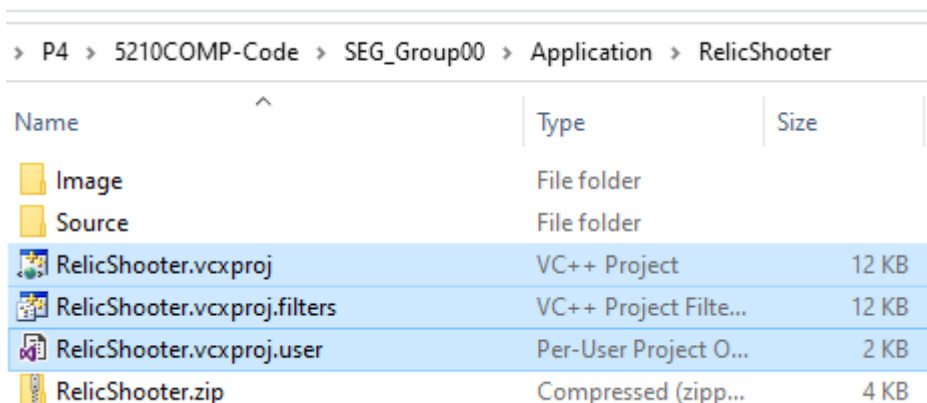
We have one more thing that we need to handle – how to allow us to work with our code in the IDE. Visual Studio generates files that are unique to each machine, so how do we handle this. If you checked out the code now and tried to run it, the project wouldn't open in Visual Studio, since it does not know about the project files. But we cannot put these on the repository, as they are unique to each machine and will cause issues with your other group members.



As we can see above, the icon shows they are not under source control

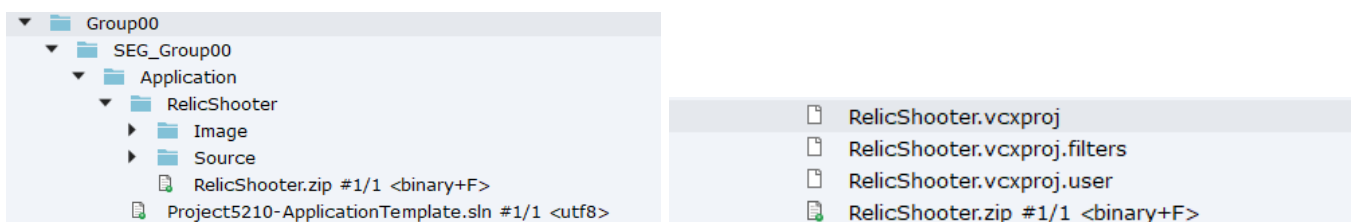
The Solution: Store them as an archive.

Locate your project in your workspace folder in Windows and find the Visual Studio files and select the three files:



Right click and use 7-Zip to archive them – my archive is called RelicShooter.zip.

Commit this to the Depot using the Mark for Add – Submit process.

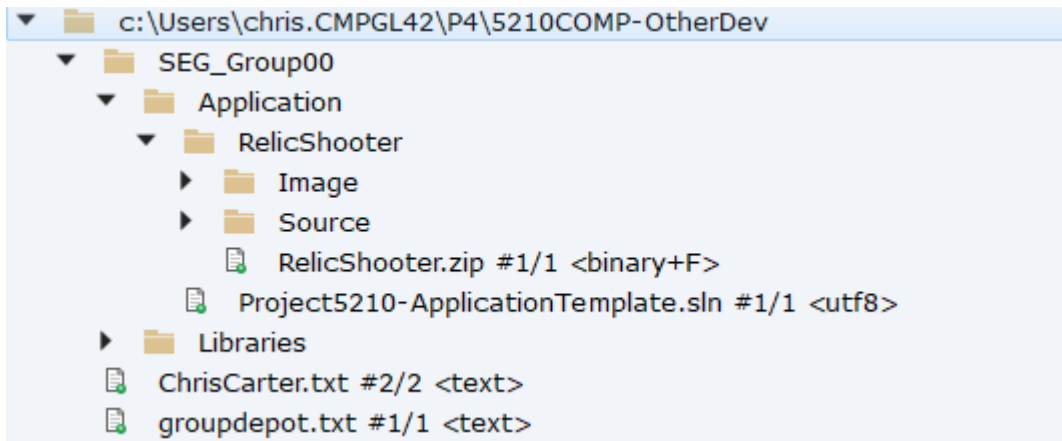


So, we can see that the Zip file is under source control, so we can easily restore these files in our workspace, but the Visual Studio files are not under source control, which means that any changes that Visual Studio makes locally to these files will not be stored on the server and therefore will not affect any one else!

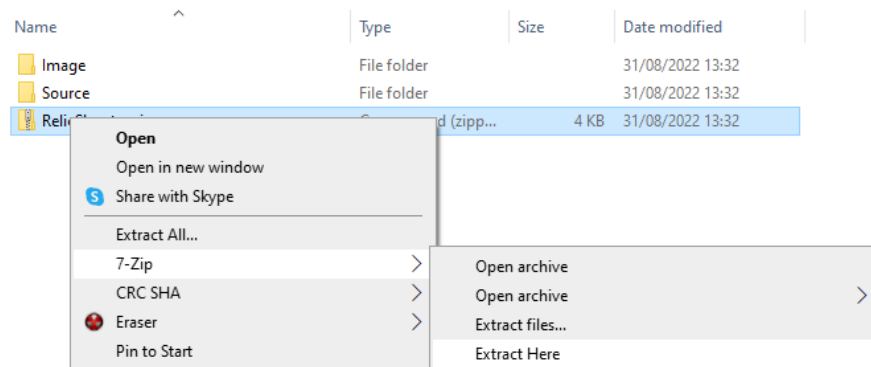
Testing each group member

So, one group member should have an up-to-date Workspace, but the rest of you will be out of sync.

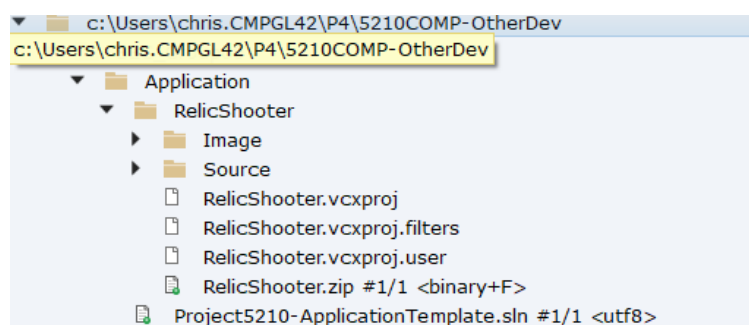
Each of the other team members should then get latest to get the code on their machine.



You can see that my other team member has the Zip file, but not the Visual Studio files. Simply go into the folder and extract the zip file via Extract Here (so no new sub folders are made):

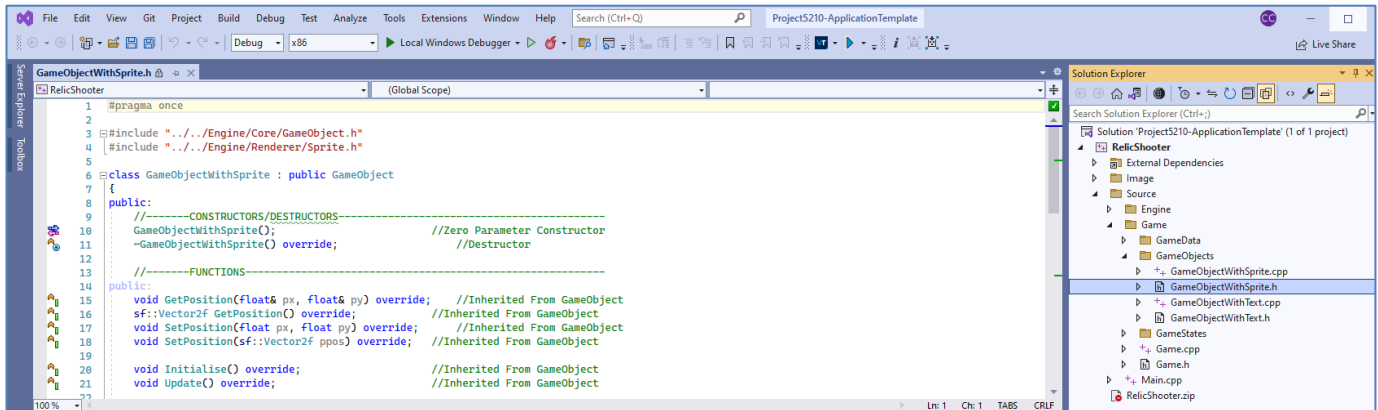


We can see that the files are in the workspace, but crucially, also not under source control so again won't affect any other user.



Now you can open the Visual Studio solution for your project:

Compile / run the project to check that it works – Always double check it is Debug – x86 – this may reset when you have new Visual Studio project files:



Golden Rules for using Perforce

So, we've covered the major aspects of working with P4V – now whenever you enter the lab, the first thing you should do is log into your workspace (Make a note of this so you don't forget!).

There are some other things that will make your lives easier when working with Source Control:

- *Try not to submit too many files in a single Commit (first time is fine) – this helps protect from any failure due to Internet connection issues.*
- *DO NOT – move, delete, or rename any files that are under source control externally, it will cause serious problems that are difficult to resolve.*
- *Always Perform these tasks via P4V.*
- *Pay close attention to the final point in the Lecture on Source Control regarding Visual Studio and read only files!! THAT IS A SHOW STOPPER FOR A LOT OF PROJECTS AND EVERY YEAR SOMEONE WILL FALL FOUL OF IT!*

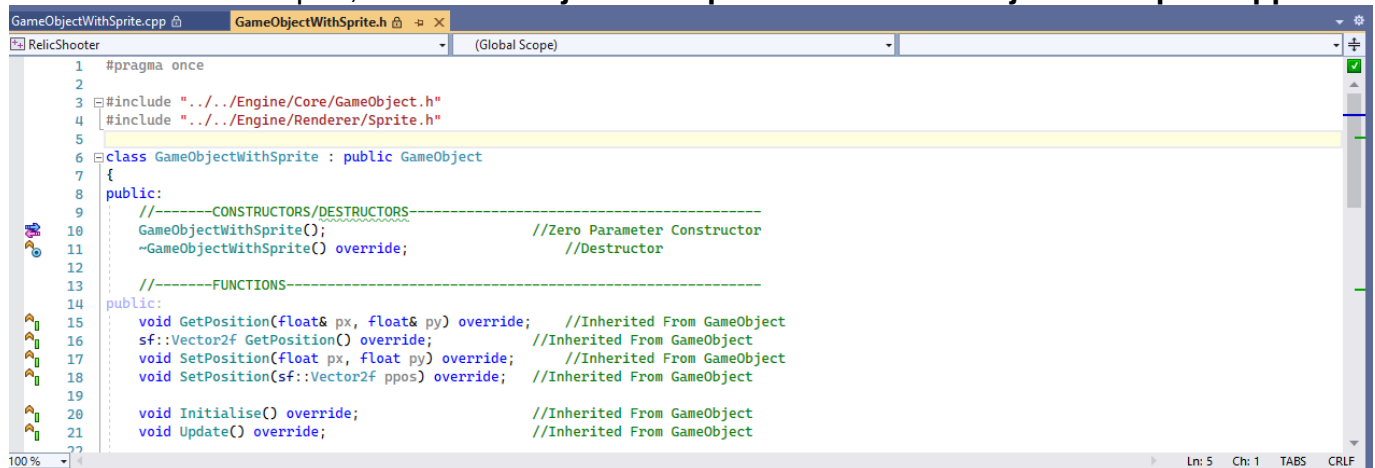
THE FINAL TASK IS TO MAKE SURE YOU CAN MODIFY CODE VIA VISUAL STUDIO CORRECTLY FOR PERFORCE.

Task 10: Checkout-Modify-Submit via Visual Studio and P4V.

The final key task that needs to be completed is to understand how to correctly work with developing new code in Visual Studio. Primarily, you will be adding new source code files when you create new classes and then modifying and updating these files to alter and engineering new code.

We will focus on the latter here.

With Visual Studio open, find **GameObjectWithSprite.h** and **GameObjectWithSprite.cpp**

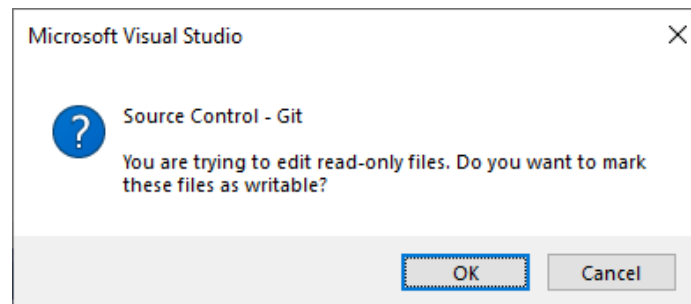


```

1  #pragma once
2
3  #include "../Engine/Core/GameObject.h"
4  #include "../Engine/Renderer/Sprite.h"
5
6  class GameObjectWithSprite : public GameObject
7  {
8  public:
9      //-----CONSTRUCTORS/DESTRUCTORS-----
10     GameObjectWithSprite();           //Zero Parameter Constructor
11     ~GameObjectWithSprite() override;  //Destructor
12
13     //-----FUNCTIONS-----
14
15     void GetPosition(float& px, float& py) override; //Inherited From GameObject
16     sf::Vector2f GetPosition() override;             //Inherited From GameObject
17     void SetPosition(float px, float py) override;   //Inherited From GameObject
18     void SetPosition(sf::Vector2f ppos) override;    //Inherited From GameObject
19
20     void Initialise() override;                      //Inherited From GameObject
21     void Update() override;                          //Inherited From GameObject
  
```

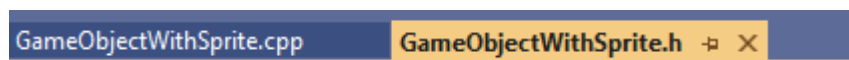
Both files have a Padlock icon showing they are read-only and therefore cannot be overwritten.

Try to add an enter into the file (e.g., add a blank line before the word **class**)



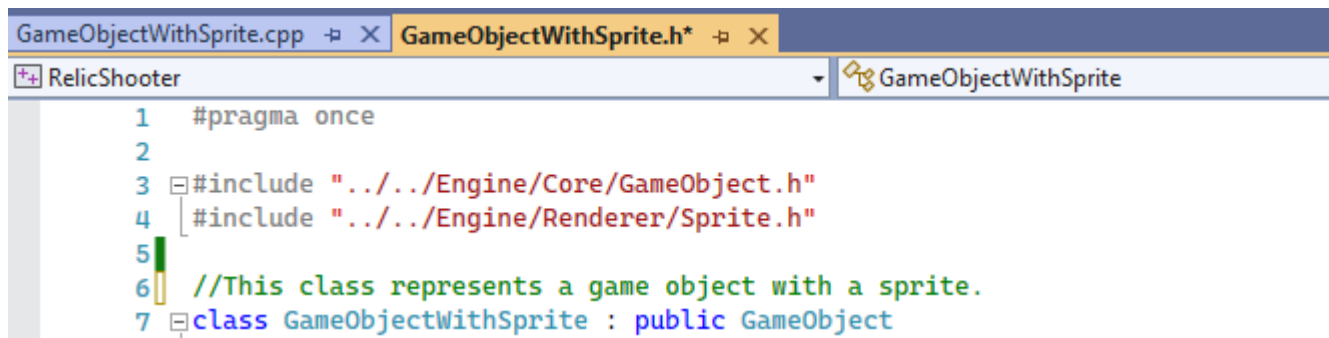
This will tell you that the file is read-only. But clicking OK will change this status so that it is modifiable. As far as perforce is concerned, this file is not being edited, so by changing this status and altering the file, perforce cannot track this and causes major issues.

Therefore, click Cancel and open P4V. Locate the .h and .cpp files and perform a checkout.



The padlock icon should disappear and allow you to complete the change.

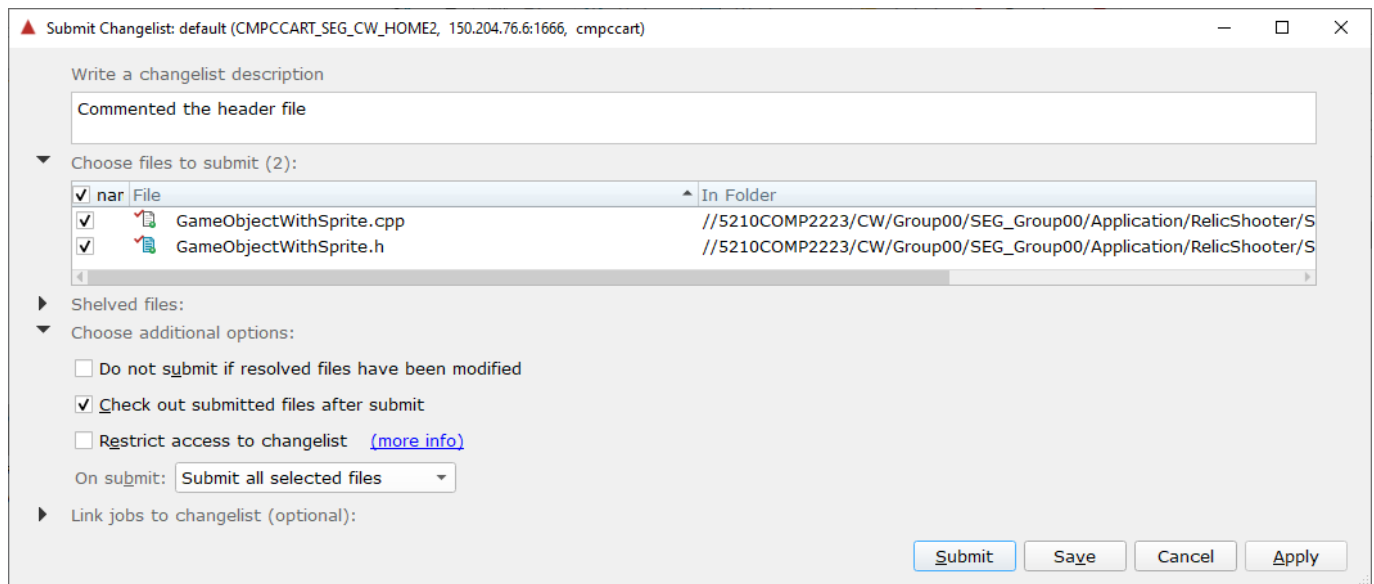
- Add in a comment line (e.g. //This class represents a game object with a sprite).



```
1 #pragma once
2
3 #include "../Engine/Core/GameObject.h"
4 #include "../Engine/Renderer/Sprite.h"
5
6 //This class represents a game object with a sprite.
7 class GameObjectWithSprite : public GameObject
```

The asterisk symbol denotes that the file has unsaved changes – It is suggested you use the File – Save All menu (Ctrl + Shift + S on the keyboard) to save any changes that are made.

- Go back to P4V and submit the change with a change description as to what has changed.



Note that under the additional options you can choose the file to be checked out again so you can then work on it again. If you submitted without this option, then the files will return to read only and you have to check them back out again to work on them.

This is the cycle of development using source control – the key advice is to commit little and often – small changes. Bigger changes merit the use of the diff tool and a code review.

- Make sure that everyone else sees your change by doing a Get Latest.

Additional Task : How to Handle Conflicts

NOTE: This task is more for awareness and reference should you encounter conflicting files – this will show you how to spot conflicts and how to resolve them.

From time to time another programmer may be editing the same file as you at the same time. If they submit their changes before you, you will have to resolve any conflicts. If another programmer submits a file that you have already checked out, this will be indicated with a yellow triangle icon:



ChrisCarter.txt #2/2 <text>

This icon indicates that we do not have the latest version of this file.

Step 1: Get Latest

To merge the files then we must get the latest version of that file; **don't worry, this will not overwrite our changes yet!**

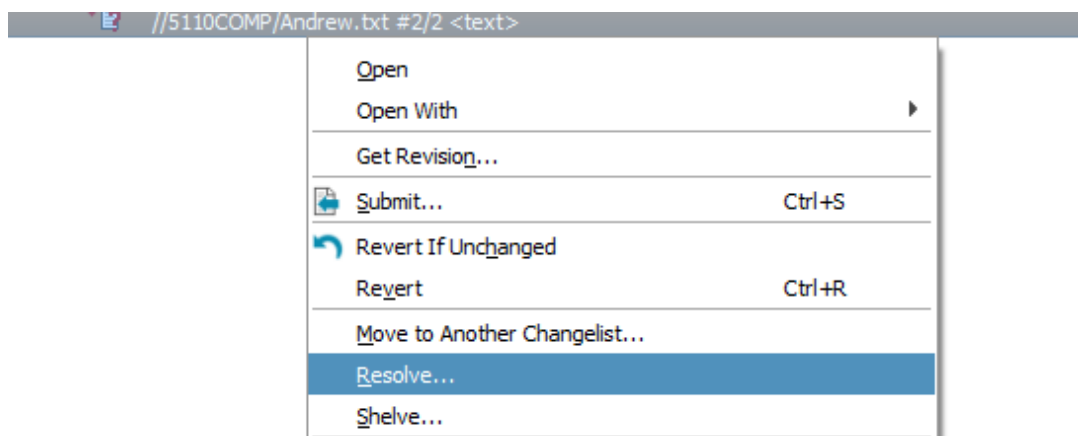
Once we have got latest then the yellow triangle will be replaced by a red question mark, indicating that you need to resolve any conflicts.



ChrisCarter.txt #2/2 <text>

Step 2: Resolving conflicts

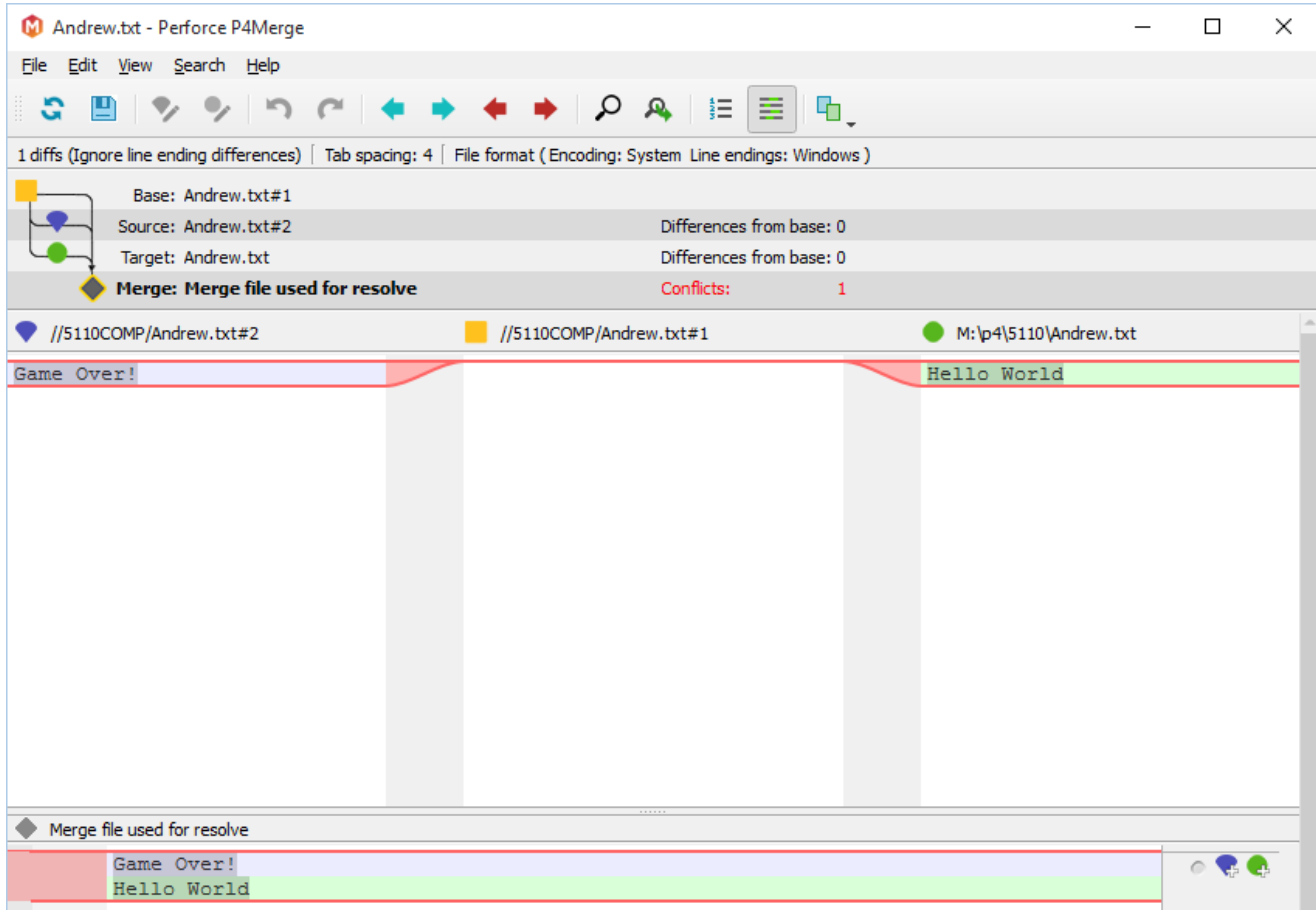
Right click on the file and select "Resolve".



You will be presented with a resolve dialogue:

This dialogue gives you a number of options in order to resolve the conflicts in the file.

The auto resolve can work for simple issues where programmers have edited different parts of the file. In this case the programmers have edited the same line so we must manually merge the files. Select the “Run Merge Tool” option.



This will run the P4 Merge tool as we saw earlier, however this time we must resolve conflicts between 3 different versions of the same file.

In the middle, indicated by the yellow square, is the common base file (the version that both programmers edited).

On the left is the updated (latest) version that has been submitted by another programmer before we have submitted our file.

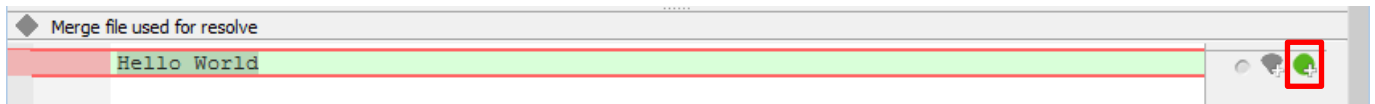
On the right is our local version of the file with the changes that we have made.

At the bottom of the window is a merged version of the file that will be the “resolved” version.

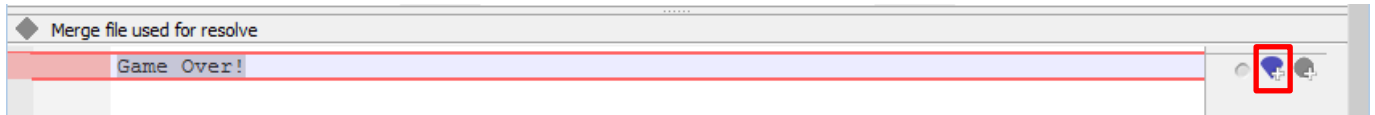
Depending on the situation, the resolved file may be fine, in this case game over has automatically been placed above Hello world.

You can accept the merged version of the file by clicking on the save icon and closing the window.

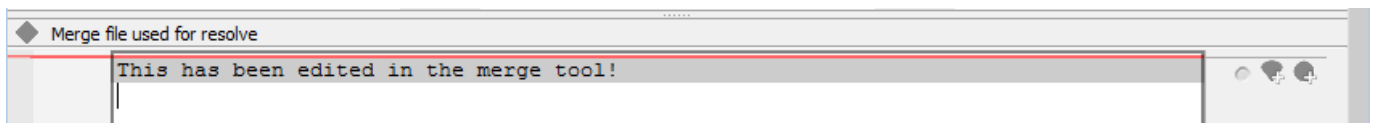
If you want to accept your version of the line, you can click the green circle to the right of the conflict:



If you want to accept their version of the line you can click on the blue pie shape to the right of the conflict:



If neither line makes sense then you can edit the merge tool by clicking on the text and editing it as you would normally:



Once you have merged all of the conflicts in the file then save the merged result by clicking the save icon and closing the P4 Merge tool.

You will be prompted to replace the workspace file with the merged result. Click yes to accept the merged result.

The red question mark will be removed from the file showing that the conflicts have been resolved.



ChrisCarter.txt #2/2 <text>

At this point, if it has been a source code file that has changed, then it is a good idea to recompile your code and test that your changes still work.

Quite frequently you will find that the merge process will have caused a compile error or runtime error in your code.

Fix any issues before submitting your code file (When working with code).